
STL

Standard Template Library

标准库模板

复习

- | | |
|-------------|----|
| 1. 模板概述 | 了解 |
| 2. 函数模板 | 重点 |
| 3. 函数模板的原理 | 熟悉 |
| 4. 函数模板的实例化 | 重点 |
| 5. 类模板 | 重点 |
| 6. 类模板实例化 | 重点 |
| 7. 模板别名 | 重点 |

目录

1. STL概述
2. STL基本组成
3. string容器
4. vector容器
5. stack容器
6. queue容器
7. deque容器
8. list容器
9. set/multiset容器
10. map/multimap容器
11. STL常用算法

了解

熟悉

重点

重点

重点

重点

重点

重点

重点

重点

重点

STL概述

STL,学名Standard Template Library, 一般称它为标准模板库。

C++ 对模板 (Template) 支持得很好, STL 就是借助模板把常用的数据结构及其算法都实现了一遍, 并且做到了数据结构和算法的分离。例如, vector 的底层为顺序表 (数组), list 的底层为双向链表, deque 的底层为循环队列, set 的底层为红黑树, hash_set 的底层为哈希表。

从根本上说, STL是一些“容器”的集合, 这些“容器”有list,vector,set,map一大堆, STL也是算法和其他一些组件的集合。比如说<algorithm>中sort函数、<string>中string类都

STL基本组成

STL从广义上讲分为三类

1. **容器 (Container)** , 是一种数据结构, 如list, vector, 和deques , 以模板类的方法提供。为了访问容器中的数据, 可以使用由容器类输出的迭代器;
2. **迭代器 (Iterator)** , 提供了访问容器中对象的方法。例如, 可以使用一对迭代器指定list或vector中的一些范围的对象。迭代器就如同一个指针。事实上, C++的指针也是一种迭代器。但是, 迭代器也可以是那些定义了operator*()以及其他类似于指针的操作符地方法的类对象;
3. **算法 (Algorithm)** , 是用来操作容器中的数据的模板函数。例如, STL用sort()来对一个vector中的数据进行排序, 用find()来搜索一个list中的对象, 函数本身与他们操作的数据的结构和类型无关, 因此他们可以在从简单数组到高度复杂容器的任何数据结构上使用;

string容器

用字符数组存放字符串容易发生数组越界的错误，而且往往难以察觉。因此，C++ 标准模板库设计了 string 数据类型，专门用于字符串处理。

要使用 string 对象，须包含头文件 `<string>`。

在用 C++ 编程时，要优先考虑用 string 对象来处理字符串，因为其用法比字符数组更简单，而且不容易出错。

string容器之初始化

string对象的初始化和普通类型变量的初始化基本相同，只是string作为类，还有类的一些特性：使用构造函数初始化。

string s1;	//默认初始化，一个空字符串s1
string s2(s1);	//s2是s1的副本
string s3 = s1;	//等价于s3 (s1) ,s3是s1的副本
string s4("hello");	//s4是字面值"hello",
string s5 = "hello";	//等价于上行代码
string s6(n,'a');	//把s6初始化为由连续的n个字符c组成的
串	

string容器之初始化

调用string的构造函数生成一个临时的string类，再用临时的string类初始化。

```
string s7 = string("hello");
```

```
string s8(string("hello"));
```

string容器之输入输出

1.cin 输入

```
string s1;
```

```
cin >> s1; //遇到空格终止
```

```
cout << s1 << endl;
```

2.getline读取整行

```
string s1;
```

```
getline(cin,s1);
```

```
cout << s1 << endl;
```



vs里面使用getline, 须加上
#include <string>

string容器之比较大小

string可以直接使用 > ,< == 等进行比较。

```
string s1 = "abc",s2="eda";
```

```
s1 > s2
```

```
s1 < s2
```

```
s1 == s2
```

string容器之链接

string可以使用+来直接链接。

```
string s1 = "hello",s2 = " world";
```

```
string s3 = s1 + s2;
```

string容器之获取字符

1.使用C++11新特性的for

```
string s1 = "abcdefg";
```

```
for (auto c:s1){
```

```
    cout << c << " ";
```

```
}
```

2.使用运算符[] + size()函数

```
s1[0];
```

```
s1[1];
```

```
s1[2];
```

string容器之获取字符

3.使用迭代器

```
string s1 = "hello world";  
for(auto i=s1.begin();i!=s1.end();i++){  
    cout << *i << " ";  
}
```

string容器之拷贝string对象

`string s(s1,pos)`

`string s(s1,pos,len)`

第一个将s1从下标pos开始拷贝到结尾。当`pos>s1.size()`时，为未定义行为；当`pos=s1.size()`，拷贝一个空字符

第二个将s1从下标pos开始拷贝，拷贝len个字符。当`pos>s1.size()`时，为未定义行为；当`pos=s2.size()`，拷贝一个空字符

`string s1("hello");`

(1) `string s2(s1, 1);`//s2为" ello",长度为4

(2) `string s3(s1, 5);`//s3为"",长度为0

(3) `string s8(s1, 6);`// 错误，未定义的行为，抛出异常

(4) `string s4(s1, 1,3);`// s4为" ell",长度为3

(5) `string s5(s1, 1,8);`// 正确， s5为" ello",长度为4

(6) `string s6(s1, 5,8);`// s6为"",长度为0

(7) `string s7(s1, 6,1);`// 错误，未定义的行为，抛出异常

string容器之substr函数

s.substr(pos,n)

返回一个string对象，返回的对象包含s从pos下标开始的n个字符。pos和n均为可选参数。pos默认为下标0；n默认为s.size()-pos。

```
string s ("value");
```

```
(1)string s2 = s.substr();//s2为"value",大小为5
```

```
(2)string s3 = s.substr(2);//s3为"lue",大小为3
```

```
(3)string s4 = s.substr(5);//s3为"",大小为0
```

```
(4)string s5 = s.substr(6);//错误, s5的大小为pos = 5, 小于s.size()
```

```
(5)string s6 = s.substr(1,2);// s6为"al",大小为2
```

```
(6)string s7 = s.substr(1,7);// s7为"alue",大小为4
```

```
(7)string s8 = s.substr(5,7);// s8为"",大小为0
```

```
(8)string s9 = s.substr(6,7);// 错误, s9的大小为pos = 5, 小于s.size()
```

string容器之insert函数

1. iterator insert(iterator pos, CharT ch)
2. void insert(iterator pos, size_type count, CharT ch)
3. void insert(iterator pos, InputIt first, InputIt last)
4. 插入初始化列表

```
string s1("value");
```

```
s1.insert(s1.begin(), 's');//执行后, s1为"svalue"
```

```
s1.insert(s1.begin(), 1, 's');//执行后, s1为"ssvalue"
```

```
s1.insert(s1.begin(), s1.begin(), ++s1.begin());//执行后, s1为"sssvalue"
```

```
s1.insert(s1.end(), {'1','2'});//执行后, s1为"sssvalue12"
```

string容器之erase函数

`basic_string & erase(size_type pos=0, size_type n=npos)`

删除s从pos下标开始的n个字符，并返回删除后的s。当pos > s.size()时，报错

`iterator erase(const_iterator position)`

删除s迭代器position位置的字符，并返回下一个字符的迭代器。

`iterator erase(const_iterator first, const_iterator last)`

删除s迭代器[first,last)区间的字符，并返回last字符的迭代器。

```
string s1("value");
```

```
string s2("value");
```

```
string s3("value");
```

```
string s4("value");
```

```
s1.erase();//执行后，s1为空
```

```
s2.erase(0,2); //执行后，s2为"lue"
```

```
s3.erase(s3.begin());//执行后，s3为"alue"
```

```
s4.erase(s4.begin(),++s4.begin());//执行后，s4为"alue"
```

string容器之append函数

append是在string对象的末尾进行插入操作。这一点使用+运算符也能做到.

```
string s="C++";
```

```
s.append("program");//执行完后, s="C++program"
```

string容器之replace函数

```
string s("i very love China!");
```

```
const char* cp1 = "truly";
```

```
const char* cp2 = "truly!!!";
```

```
string str1 = "really";
```

```
string str2 = "really!!!";
```

//1.将s从下标2开始删除4个字符，删除后在下标2处插入cp1

```
s.replace(2,4,cp1);//s=" i truly love China!"
```

//2.将s从下标2开始删除5个字符，删除后在下标2插入cp2的前5个字符

```
s.replace(2, 5, cp2,5); //s=" i truly love China!"
```

//3.将s从下标2开始删除5个字符，删除后在下标2插入str1

```
s.replace(2, 5, str1);//s="i really love China!"
```

//4.将s从下标2开始删除6个字符，删除后在下标2插入str2从下标0开始的6个字符

```
s.replace(2, 6, str2,0,6);//s="i really love China!"
```

//5.将s从下标2开始删除6个字符，删除后在下标2插入4个'*'字符

```
s.replace(2, 6, 4, '*');//s="i **** love China!"
```

string容器之其他函数

string的相关操作函数太多太多。

可以观看前面视频。这里就不一个一个列举了。

接下来看个练习案例

string 练习 题目

输入一串带有标点符号的字符串，去除字符串中的标点符号！



判断字符是否是标点符号用
`ispunct()`

string 练习 代码

```
#include <iostream>
```

```
#include <cstring>
```

```
#include <sstream>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
    string s;
```

```
    cin >> s;
```

```
    for (auto each : s) {
```

```
        if (!ispunct(each)) {
```

```
            cout << each;
```

```
        }
```

```
    }
```

```
    return 0;
```

```
}
```

vector容器

c++ 中，vector 是一个十分有用的容器。它能够像容器一样存放各种类型的对象。也就是说，vector是一个能够存放任意类型的动态数组。

vector 是同一种类型的对象的集合，每个对象都有一个对应的整数索引值。和 string 对象一样，标准库将负责管理与存储元素相关的内存。

把 vector 称为容器，是因为它可以包含其他对象。一个容器中的所有对象都必须是同一种类型的。

vector容器之头文件

使用vector须包含头文件vector

```
#include <vector>
```

vector容器之声明与初始化

vector 可以声明各种类型，整型、字符型、字符串等等！

//整形数组

```
vector <int> vec = { 1,2,3,4,5};
```

//char型数组

```
vector <char> vec1 = {'h','e','l','l','o' };
```

//double

```
vector<double> vec2 = {1.1,2.2,3.3};
```

vector容器之声明与初始化

利用构造函数初始化

`vector()`:创建一个空vector `vector(int nSize)`:创建一个vector,元素个数为nSize

`vector(int nSize,const t& t)`:创建一个vector, 元素个数为nSize,且值均为t

`vector(const vector&)`:复制构造函数

`vector(begin,end)`:复制[begin,end)区间内另一个数组的元素到vector中

`vector<int> vec(10);`//定义一个含10个变量的整型数组vec, 默认值都为0

`vector<int> vec(10,2);`//定义一个含10个变量的整型数组vec, 默认值都为2

`vector<int> vec(a);`//其中a也是vector, 把a整体赋值给vec

`vector<int> vec(a.begin(),a.begin+1);`//把a的从开始到第二个赋值给vec

vector容器之遍历函数

reference at(int pos):返回pos位置元素的引用

reference front():返回首元素的引用

reference back():返回尾元素的引用

iterator begin():返回向量头指针，指向第一个元

iterator end():返回向量尾指针，指向向量最后一个元素的下一个位置

reverse_iterator rbegin():反向迭代器，指向最后一个元素

reverse_iterator rend():反向迭代器，指向第一个元素之前的位置

vector容器之遍历函数

```
vector<int> v = { 1,2,3,4,5,6 };  
cout << v.at(1) << endl; // 打印2  
cout << v.front() << endl; //打印1  
cout << v.back() << endl; // 打印6
```

```
for (auto i = v.begin(); i != v.end(); i++) {  
    cout << *i << " ";  
}  
//输出 1 2 3 4 5 6  
cout << endl;  
for (auto i = v.rbegin(); i != v.rend(); i++) {  
    cout << *i << " ";  
}  
//输出 6 5 4 3 2 1
```

vector容器之增加函数

`void push_back(const T& x)`
//向量尾部增加一个元素x

`iterator insert(iterator it, const T& x)`
//向量中迭代器指向元素前增加一个元素x

`iterator insert(iterator it, int n, const T& x)`
//向量中迭代器指向元素前增加n个相同的元素x

`iterator insert(iterator it, const_iterator first, const_iterator last)`
//向量中迭代器指向元素前插入另一个相同类型向量的[first,last)间的数据

vector容器之增加函数

```
vector<int> v = { 1,2 };  
vector<int> v2 = { 100,200 };  
v.push_back(3);  
v.insert(v.begin(), 11);  
v.insert(v.begin() + 1, 3, 22);  
v.insert(v.begin(), v2.begin(), v2.end());  
  
for (auto i = v.begin(); i != v.end(); i++) {  
    cout << *i << " ";  
}
```

Microsoft Visual Studio 调试控制台

```
100 200 11 22 22 22 1 2 3  
E:\UStudioCode\Test001\x64\Debug\Test0  
按任意键关闭此窗口. . .
```

vector容器之删除函数

iterator erase(iterator it):删除向量中迭代器指向元素

iterator erase(iterator first,iterator last):删除向量中[first,last)中元素

void pop_back():删除向量中最后一个元素

void clear():清空向量中所有元素

vector容器之删除函数

```
vector<int> v = { 1,2,3,4,5,6 };
```

```
    v.erase(v.begin()+1); //删除2
```

```
    v.erase(v.begin() + 2, v.begin() + 4); //删除 4 , 5
```

```
    v.pop_back(); //删除最后一个元素
```

```
    v.clear(); //清空所有元素
```

```
    for (auto i = v.begin(); i != v.end(); i++) {
```

```
        cout << *i << " ";
```

```
    }
```

vector容器之判断函数

`bool empty() const`:判断向量是否为空，若为空，则向量中无元素

```
cout << v.empty() << endl;
```

vector容器之大小函数

`int size() const`:返回向量中元素的个数

`int capacity() const`:返回当前向量所能容纳的最大元素值

`int max_size() const`:返回最大可允许的 vector 元素数量值

vector容器之大小函数

```
vector<int> v = { 1,2,3,4,5,6 };
```

```
cout << v.size() << endl;
```

```
cout << v.capacity() << endl;
```

```
cout << v.max_size() << endl;
```

Microsoft Visual Studio 调试控制台

6

6

4611686018427387903

E:\UStudioCode\Test001\x64\Debug\Test

按任意键关闭此窗口. . .

vector容器之其他函数

`void swap(vector&)`

交换两个同类型向量的数据

`void assign(int n,const T& x)`

设置向量中前n个元素的值为x

`void assign(const_iterator first,const_iterator last)`

向量中[first,last)中元素设置成当前向量元素

vector容器之其他函数

```
vector<int> v;  
v.assign(4,2); //是a只含4个元素，且每个元素为2
```

```
vector<int> v1 = { 1,2,3,4,5,6};  
vector<int> v2 = { 11,22,33 };  
v2.assign(v1.begin(), v1.end());  
for (auto i = v2.begin(); i != v2.end(); i++) {  
    cout << *i << " ";  
}  
//v2里面变成1,2,3,4,5,6
```

```
vector<int> v1 = { 1,2,3,4,5,6};  
vector<int> v2 = { 11,22,33 };  
v1.swap(v2); //交换v1和v2  
for (auto i = v1.begin(); i != v1.end(); i++) {  
    cout << *i << " ";  
}  
cout << endl;  
for (auto i = v2.begin(); i != v2.end(); i++) {  
    cout << *i << " ";  
}
```

案例题目

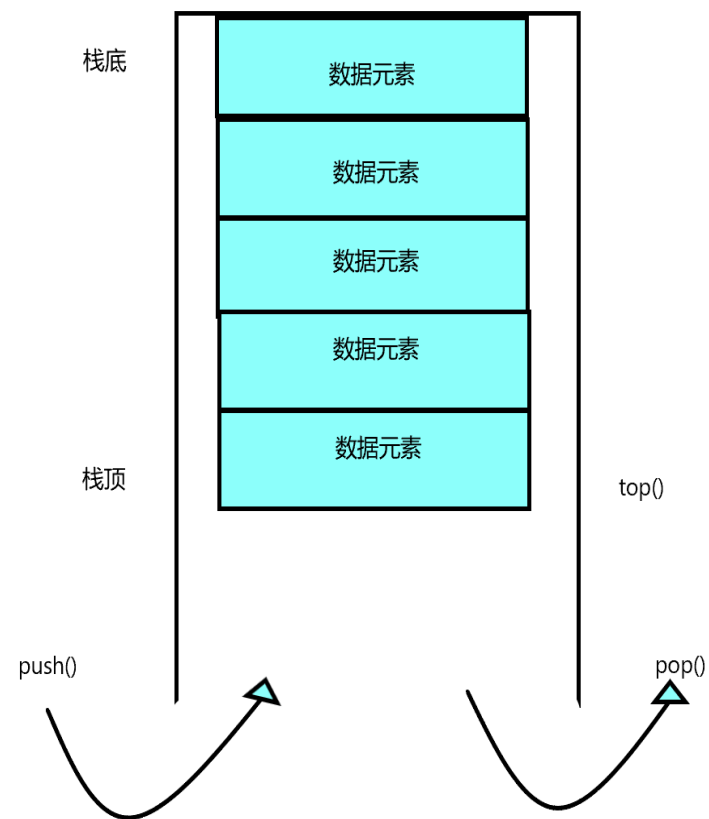
输入n，随机n个1-100的数字，装入vector，找出里面最大值和最小值。

案例代码

```
#include <iostream>
#include<cstdlib>
#include<ctime>
#include <vector>
using namespace std;
int main() {
    int n;
    cin >> n;
    vector<int> v;
    srand(time(0));
    for (int i = 1; i <= n; i++) {
        v.push_back(rand() % 100 + 1);
    }
    int max = v[0], min = v[0];
    cout << max << " " << min << endl;
    for (int i = 0; i < n; i++) {
        if (max < v[i]) {
            max = v[i];
        }
        if (min > v[i]) {
            min = v[i];
        }
    }
    cout << max << " " << min << endl;
    return 0;
}
```

stack容器

stack（栈）容器是一种先进后出的容器，两端只有一个开口，只能从这一个开口插入和删除数据。



stack容器

使用时包含头文件stack

```
#include <stack>
```

构造函数

stack<T> stkT;//stack采用模板类实现， stack对象的默认构造形式：

```
stack(const stack &stk);//拷贝构造函数
```

赋值函数

```
stack&operator=(const stack &stk);//重载等号操作符
```

stack容器之数据存取

`push(elem);`//向栈顶添加元素

`pop();`//从栈顶移除第一个元素

`top();`//返回栈顶元素

stack容器之数据存取

```
#include <iostream>

#include <stack>

using namespace std;

int main() {
    stack<int> s;
    s.push(111); //添加元素111
    s.push(222); //添加元素222
    s.push(333); //添加元素333

    s.pop(); //弹出333
    int n = s.top(); //n = 222
    return 0;
}
```

stack容器之大小操作

`empty();`//判断堆栈是否为空

`size();`//返回堆栈的大小

stack容器之大小操作

```
#include <iostream>

#include <stack>

using namespace std;

int main() {
    stack<int> s;

    s.push(111); //添加元素111
    s.push(222); //添加元素222
    s.push(333); //添加元素333


    cout << s.empty() << endl; //0
    cout << s.size() << endl; // 3
    return 0;
}
```

stack 案例题目

输入一个十进制整数，输出对应的二进制数。

stack 案例代码

```
#include <iostream>
#include <stack>
using namespace std;
int main() {
    stack<int> s;
    int n;
    cin >> n;
    while (n)//循环取余入栈到s中
    {
        s.push(n % 2);
        n /= 2;
    }
    while (!s.empty())//从栈顶去取元素
    {
        cout << s.top();
        s.pop();
    }
    return 0;
}
```

queue容器

Queue是一种先进先出（First In First Out,FIFO）的数据结构，它有两个出口。

queue模版类的定义在<queue>头文件中。

```
#include<queue>
```

定义queue对象的示例代码如下：

```
queue<int>q1;
```

```
queue<double>q2;
```

queue容器



1. 队列容器允许从一端新增元素，从另一端移除元素
2. 队列中只有对头和队尾才可以被外界使用，因此队列不允许有遍历的行为
3. 队列中进数据称为-入队 push
4. 队列中出数据称为-出队 pop

queue容器之构造函数

`queue<T> queT;`//queue采用模板类实现, queue对象的默认构造形式:
`queue(const queue &que);`//拷贝构造函数

```
queue<int> q1;  
queue<int> q2(q1);
```

queue容器之相关函数

`push(elem);`//往队尾添加元素

`pop();`//从队头移除第一个元素

`back();`//返回最后一个元素

`front();`//返回第一个元素

`empty();`//判断队列是否为空

`size();`//返回队列的大小

queue容器之相关函数

```
#include <iostream>
#include <queue>
using namespace std;
int main() {
    queue<int> q1;
    //添加元素
    q1.push(111);
    q1.push(222);
    q1.push(333);

    cout << q1.size() << endl; //输出长度

    int first = q1.front(); //返回111
    int end = q1.back(); //返回333

    q1.pop(); //移除队头元素

    while (!q1.empty()) {
        cout << q1.front() << " ";
        q1.pop();
    }

    return 0;
}
```

 Microsoft Visual Studio 调试控制台

```
222 333
E:\UStudioCode\Test001\x64\De
按任意键关闭此窗口. . .
```

queue题目

有一堆扑克牌，里面有 n 张扑克牌。第一次从牌堆顶上拿出一张牌并输出，第二次把牌放回牌堆底下。重复执行直到牌堆里没牌。也就是说奇数张的牌输出，偶数张的牌放回。

输入：

4

1 2 3 4

输出：

1 3 2 4

queue代码

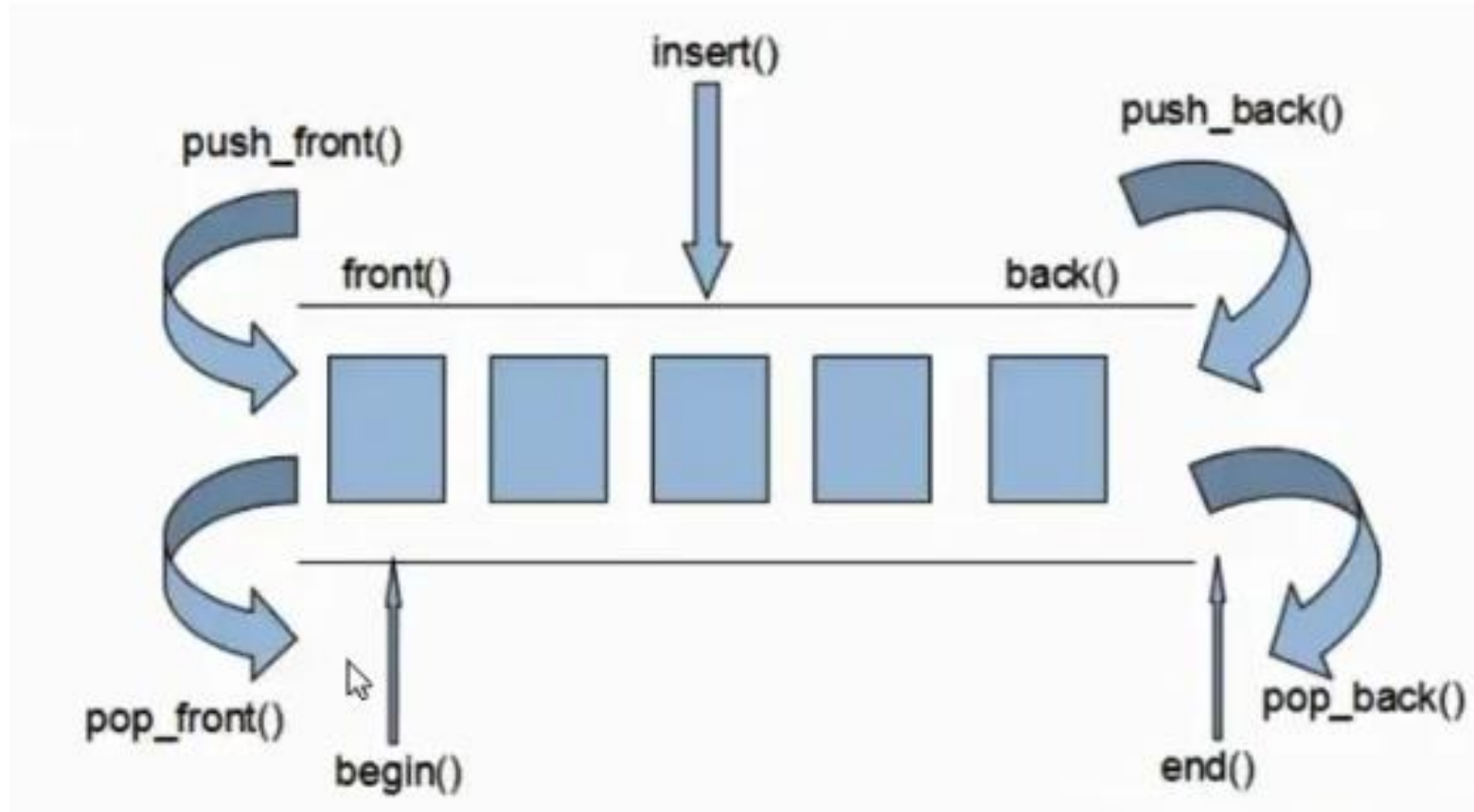
```
#include <iostream>
#include <queue>
using namespace std;
int main() {
    int n,num;
    cin >> n;
    queue <int> q;
    for (int i = 0; i < n; i++) {
        cin >> num;
        q.push(num);
    } //输入数据
    bool b = 1;
    while (!q.empty()) {
        if (b) {
            cout << q.front() << " ";
            q.pop(); //每到奇数次就为真，并输出，删掉
        }
        else {
            q.push(q.front()); //偶数次为假放回牌堆底部
            q.pop();
        }
        b = !b; //由于有两种情况，所以每次判断完之后，b返回相反值
    }
    return 0;
}
```

deque容器

deque双端队列，可以对头端和尾端进行插入删除操作。

deque队列为一个给定类型的元素进行线性处理，像向量一样，它能够快速地随机访问任一个元素，并且能够高效地插入和删除容器的尾部元素。但它又与vector不同，deque支持高效插入和删除容器的头部元素。

deque容器



deque容器之构造函数

`deque<T> deqT; //默认构造形式`

`deque(beg, end); //构造函数将[beg,end]区间中的元素拷贝给本身`

`deque(n, elem); //构造函数将n个elem拷贝给本身`

`deque(const deque &deq); //拷贝构造函数`

deque容器之构造函数

```
deque<int> a; // 定义一个int类型的双端队列a
```

```
deque<int> a(10);
```

```
// 定义一个int类型的双端队列a，并设置初始大小为10
```

```
deque<int> a(10, 1);
```

```
// 定义一个int类型的双端队列a，并设置初始大小为10且初始值都为1
```

```
deque<int> b(a);
```

```
// 定义并用双端队列a初始化双端队列b
```

```
deque<int> b(a.begin(), a.begin()+3);
```

```
// 将双端队列a中从第0个到第2个(共3个)作为双端队列b的初始值
```

deque容器之添加函数

d.push_front(const T& x);

//头部添加元素

d.push_back(const T& x);

//末尾添加元素

d.insert(iterator it, const T& x);

//任意位置插入一个元素

d.insert(iterator it, int n, const T& x);

//任意位置插入 n 个相同元素

d.insert(iterator it, iterator first, iterator last);

//插入另一个向量的 [first,last] 间的数据

deque容器之添加函数

```
#include <iostream>
#include <deque>
using namespace std;
int main()
{
    deque<int> d1;

    // 头部增加元素
    d1.push_front(4);
    // 末尾添加元素
    d1.push_back(5);
    // 任意位置插入一个元素
    deque<int>::iterator it = d1.begin();
    d1.insert(it, 2);
    // 任意位置插入n个相同元素
    it = d1.begin();
    d1.insert(it, 3, 9);
    // 插入另一个向量的[forst,last]间的数据
    deque<int> d2(5, 8);
    it = d1.begin();
    d1.insert(it, d2.end() - 1, d2.end());

    // 遍历显示
    for (it = d1.begin(); it != d1.end(); it++)
        cout << *it << " "; // 输出: 8 9 9 9 2 4 5
    cout << endl;

    return 0;
}
```

deque容器之删除函数

d1.pop_front();

//头部删除元素

d1.pop_back();

//末尾删除元素

d1.erase(iterator it);

//任意位置删除一个元素

d1.erase(iterator first, iterator last);

//删除 [first,last] 之间的元素

d1.clear();

//清空所有元素：

deque容器之删除函数

```
#include <iostream>
#include <deque>
using namespace std;
int main()
{
    deque<int> d1;
    for (int i = 0; i < 8; i++)
        d1.push_back(i);
    // 头部删除元素
    d1.pop_front();
    // 末尾删除元素
    d1.pop_back();
    // 任意位置删除一个元素
    deque<int>::iterator it = d1.begin();
    d1.erase(it);
    // 删除[first,last]之间的元素
    d1.erase(d1.begin(), d1.begin() + 1);

    // 遍历显示
    for (it = d1.begin(); it != d1.end(); it++)
        cout << *it << " ";
    cout << endl;
    // 清空所有元素
    d1.clear();
    // 遍历显示
    for (it = d1.begin(); it != d1.end(); it++)
        cout << *it << " "; // 输出: 3 4 5 6
    cout << endl;
    return 0;
}
```

deque容器之访问函数

下标访问: `deq[1];` // 并不会检查是否越界

at 方法访问: `deq.at(1);`

// 以上两者的区别就是 at 会检查是否越界, 是则抛出 `out of range` 异常

访问第一个元素: `deq.front();`

访问最后一个元素: `deq.back();`

deque容器之访问函数

```
#include <iostream>
#include <deque>
using namespace std;
int main()
{
    deque<int> d1;
    for (int i = 0; i < 6; i++){
        d1.push_back(i);
    }
    // 下标访问
    cout << d1[0] << endl; // 输出： 0
    // at方法访问
    cout << d1.at(0) << endl; // 输出： 0
    // 访问第一个元素
    cout << d1.front() << endl; // 输出： 0
    // 访问最后一个元素
    cout << d1.back() << endl; // 输出： 5

    return 0;
}
```

deque容器之容量函数

d.size();

//容器大小

d.max_size();

//容器最大容量

d.resize();

//更改容器大小

deq.empty();

//容器判空

deq.shrink_to_fit();

//减少容器大小到满足元素所占存储空间的大小

deque容器之容量函数

```
#include <iostream>
#include <deque>
using namespace std;
int main()
{
    deque<int> d1;
    for (int i = 0; i < 6; i++)
    {
        d1.push_back(i);
    }

    cout << d1.size() << endl; // 输出： 6
    cout << d1.max_size() << endl; // 输出： 1073741823
    d1.resize(0); // 更改元素大小
    cout << d1.size() << endl; // 输出： 0
    if (d1.empty()){
        cout << "元素为空" << endl; // 输出： 元素为空
    }
    return 0;
}
```

deque容器之其他函数

`deq.assign(int nSize, const T& x);` //多个元素赋值
// 类似于初始化时用数组进行赋值

`swap(deque&);`//交换两个同类型容器的元素

deque容器之其他函数

```
#include <iostream>
#include <deque>
using namespace std;
int main(){
    // 多个元素赋值
    deque<int> d1;
    d1.assign(3, 1);
    deque<int> d2;
    d2.assign(3, 2);

    // 交换两个容器的元素
    d1.swap(d2);

    // 遍历显示
    cout << "d1: ";
    for (int i = 0; i < d1.size(); i++)
        cout << d1[i] << " "; // 输出: 2 2 2
    cout << endl;

    // 遍历显示
    cout << "d2: ";
    for (int i = 0; i < d2.size(); i++)
        cout << d2[i] << " "; // 输出: 1 1 1
    cout << endl;

    return 0;
}
```

deque容器 练习题目

请设计一个排队程序，用户有普通客人和VIP客人之分，VIP客人不排队（即VIP客人在队列头部），请将已有的guest1和guest2放入队列中（guest1排在guest2前），并将VIP客人新增至队列头部。

输入描述：

无

输出描述：

VIP客人姓名 guest1姓名 guest2姓名 （每个客人的名字用空格隔开）

deque容器 练习代码

```
#include <iostream>

#include <deque>

using namespace std;

class Guest {
public:
    string name;
    bool vip;

    Guest(string name, bool vip) {
        this->name = name;
        this->vip = vip;
    }
};
```

```
int main() {

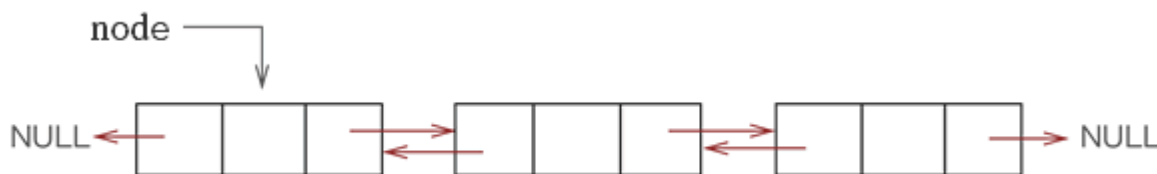
    Guest guest1("小明", false);
    Guest guest2("小华", false);
    Guest vipGuest("小张", true);
    deque<Guest> deque;
    deque.push_front(vipGuest);
    deque.push_back(guest1);
    deque.push_back(guest2);

    for (Guest g : deque) {
        cout << g.name << " ";
    }

    return 0;
}
```

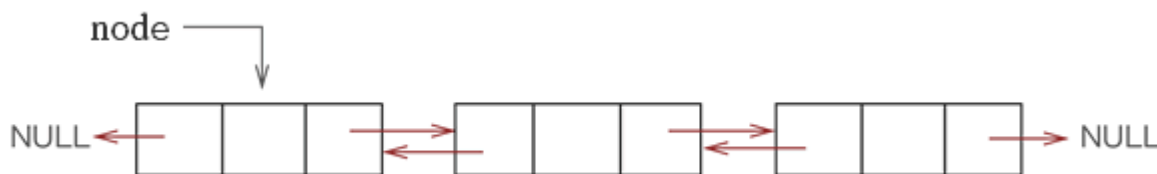
list容器

list是一种序列式容器。list容器完成的功能实际上和数据结构中的双向链表是极其相似的，list中的数据元素是通过链表指针串连成逻辑意义上的线性表，也就是list也具有链表的主要优点，即：在链表的任一位置进行元素的插入、删除操作都是快速的。



list容器

list是一种序列式容器。list容器完成的功能实际上和数据结构中的双向链表是极其相似的，list中的数据元素是通过链表指针串连成逻辑意义上的线性表，也就是list也具有链表的主要优点，即：在链表的任一位置进行元素的插入、删除操作都是快速的。



list容器之初始化

`list<int> l1;` //创建一个空链表

`list<int> l2(10);` //创建一个链表其有10个空元素

`list<int> l3(5,20);` //创建一个链表其有5个元素内容为20

`list<int> l4(l3.begin(),l3.end());` //创建一个链表其内容为l3的内容

`list<int> l5(l4);` //创建一个链表其内容为l4的内容



`#include<list>`

list容器之迭代器

遍历元素

```
list<int> li = {1,2,3,4,5,6};  
for(list<int>::iterator it=li.begin(); it!=li.end(); it++){  
    cout<<*it<<' '  
}  

```



也可以使用auto

list容器之常用函数

empty()

返回一个bool类型的值，只存在真和假，当链表为空时为真，不为空时为假

```
if(li.empty()){    //当链表为空的时候执行
    cout<<"is empty()"<<endl;
}else{
    cout<<"not empty()"<<endl;
}
```

list容器之常用函数

size()

返回链表元素的个数

```
cout<<li.size()<<endl;
```

list容器之常用函数

链表前插入push_front()

删除 pop_front()

push_front()表示在链表最前端插入一个数据

pop_front()表示在链表最前端删除一个数据。

```
li.push_front(111);
```

```
li.pop_front();
```

list容器之常用函数

链表后插入push_back()

删除 pop_back()

push_back()表示在链表尾插入一个数据

pop_back()表示将链表尾删除一个数据。

```
li.push_back(222);
```

```
li.pop_back();
```

list容器之常用函数

插入insert()

插入元素到指定位置，通过在元素之前在指定位置插入新元素来扩展向量，从而有效地增加容器大小所插入的元素数量。

插入单一数据到指定位置：

```
iterator insert (iterator position, const value_type& val);
```

插入一段数据到指定位置：

```
void insert (iterator position, size_type n, const value_type& val);
```

插入一段别的容器的数据到指定位置：

```
template <class InputIterator>
```

```
void insert (iterator position, InputIterator first, InputIterator last);
```

list容器之常用函数

`li.insert(li.begin(),100);` //在链表最前端插入数据100

`li.insert(li.begin(),3,200);` //在链表最前端插入3个数据内容为200

`list<int> k(2,50);` //创建一个新的链表k,其拥有2个元素内容均为50

`li.insert(li.begin(),li.begin(),li.end());` //在链表v最前端插入链表上k的全部内容

list容器之常用函数

erase()

删除一个元素，或者是一段区间的元素，将会自动缩减空间使用。

iterator erase (iterator position);

iterator erase (iterator first, iterator last);

```
li.erase(li.begin()); //删除
```

```
li.erase(li.begin(),li.begin()+4); //删除前4个元素
```

list容器之常用函数

排序sort()

让整个链表变成升序状态，或者变成自定义的排序状态

```
template <class Compare> void sort (Compare comp);
```

list容器之常用函数

```
#include<iostream>
#include<list>
using namespace std;
int cmp(const int &a,const int &b){
    return a>b;//简单的自定义降序序列
}

int main(){
    list<int> li;        //创建一个空链表
    for(int i=10;i>=6;i--){
        li.push_back(i);
    }
    li.push_front(3);
    li.push_back(20);
    list<int> li2(li);
    for(list<int>::iterator it=li.begin();it!=li.end();it++){
        cout<<*it<<' ';
    }
    cout<<endl;
    //排序前3 10 9 8 7 6 20//
    li.sort();
    for(list<int>::iterator it=li.begin();it!=li.end();it++){
        cout<<*it<<' ';
    }
    cout<<endl;
    //默认排序后 3 6 7 8 9 10 20//
    li2.sort(cmp);
    for(list<int>::iterator it=li2.begin();it!=li2.end();it++){
        cout<<*it<<' ';
    }
    cout<<endl;
    //自定义排序后 20 10 9 8 7 6 3//
    return 0;
}
```

list容器之常用函数

reverse()

相对于自定义的降序方法，STL提供了一个默认的降序方法reverse()，类似于sort一样直接使用即可。

```
li.reverse();
```

list容器函数列表

Lst1.assign() 给list赋值

Lst1.back() 返回最后一个元素

Lst1.begin() 返回指向第一个元素的迭代器

Lst1.clear() 删除所有元素

Lst1.empty() 如果list是空的则返回true

Lst1.end() 返回末尾的迭代器

Lst1.erase() 删除一个元素

Lst1.front() 返回第一个元素

Lst1.get_allocator() 返回list的配置器

Lst1.insert() 插入一个元素到list中

Lst1.max_size() 返回list能容纳的最大元素数量

Lst1.merge() 合并两个list

list容器函数列表

Lst1.pop_back() 删除最后一个元素

Lst1.pop_front() 删除第一个元素

Lst1.push_back() 在list的末尾添加一个元素

Lst1.push_front() 在list的头部添加一个元素

Lst1.rbegin() 返回指向第一个元素的逆向迭代器

Lst1.remove() 从list删除元素

Lst1.remove_if() 按指定条件删除元素

Lst1.rend() 指向list末尾的逆向迭代器

Lst1.resize() 改变list的大小

Lst1.reverse() 把list的元素倒转

Lst1.size() 返回list中的元素个数

Lst1.sort() 给list排序

Lst1.splice() 合并两个list

Lst1.swap() 交换两个list

Lst1.unique() 删除list中重复的元素

list 练习题目

利用list将Person自定义数据类型进行排序，Person中属性有姓名、年龄、身高。
排序规则，按照年龄进行升序，如果年龄相同按照身高进行降序。

list 练习代码1

```
#include <iostream>
#include <list>
#include <string>
using namespace std;
class Person
{
public:
    Person(string name, int age, int height)
    {
        this->m_age = age;
        this->m_height = height;
        this->m_name = name;
    }
    int m_age;
    int m_height;
    string m_name;
};
bool compare(Person& p1, Person& p2)
{
    if (p1.m_age == p2.m_age) {
        return p1.m_height > p2.m_height;//年龄相同身高降序
    }
    else {
        return p1.m_age < p2.m_age;
    }
}
```

list 练习代码2

```
void test1()
{
    list<Person> L;
    Person p1("唐僧", 35, 175);
    Person p2("孙悟空", 45, 100);
    Person p3("猪八戒", 40, 170);
    Person p4("沙悟净", 40, 190);

    L.push_back(p1);
    L.push_back(p2);
    L.push_back(p3);
    L.push_back(p4);

    cout << "排序前: " << endl;
    for (list<Person>::iterator it = L.begin(); it != L.end(); it++) { //遍历一遍
        cout << "姓名:" << (*it).m_name << " 年龄:" << (*it).m_age << " 身高:" << (*it).m_height << endl;
    }
    L.sort(compare);
    cout << "-----" << endl;
    cout << "排序后: " << endl;
    for (auto it = L.begin(); it != L.end(); it++) {
        cout << "姓名:" << (*it).m_name << " 年龄:" << (*it).m_age << " 身高:" << (*it).m_height << endl;
    }
}

int main(void)
{
    test1();
    return 0;
}
```

set/multiset容器

set/multiset属于关联式容器，底层结构使用二叉树实现的。

set/multiset的特点是：所有的元素在插入时会自动被排序。

而set与multiset容器的区别就是：set容器中不允许有重复的元素，而multiset允许容器中有重复的元素。

set构造和赋值

`set<T> st` 默认构造函数

`set(const set &st)` 拷贝构造函数

`set& operator=(const set &st)` 重载等号操作符

set构造和赋值

```
#include<iostream>
#include<set>
using namespace std;
int main() {
    set<int> s1;
    //该容器插入数据只有insert方式
    s1.insert(10);
    s1.insert(40);
    s1.insert(30);
    s1.insert(20);
    s1.insert(30);
    //遍历容器
    //所有的元素在插入的时候会自动升序排序
    //set容器不允许插入重复值
    //打印10 20 30 40
    for (auto it = s1.begin(); it != s1.end(); it++) {
        cout << *it << " ";
    }
    cout << endl;
    //拷贝构造
    set<int> s2(s1);
    for (auto it = s2.begin(); it != s2.end(); it++) {
        cout << *it << " ";
    }
    cout << endl;

    //赋值
    set<int> s3;
    s3 = s2;
    for (auto it = s3.begin(); it != s3.end(); it++) {
        cout << *it << " ";
    }
    cout << endl;
    return 0;
}
```

set容器插入与删除

<code>insert(elem)</code>	在容器中插入元素
<code>clear()</code>	清除所有元素
<code>erase(pos)</code>	删除pos迭代器所指的元素，返回下一个元素的迭代器
<code>erase(beg,end)</code>	删除区间[beg,end)的所有元素，返回下一个元素的迭代器
<code>erase(elem)</code>	删除容器中值为elem的元素

set容器插入与删除

```
#include<iostream>
#include <set>
using namespace std;
void printSet(set<int>& s) {
    for (auto it = s.begin(); it != s.end(); it++) {
        cout << *it << " ";
    }
    cout << endl;
}
//set插入与删除
int main() {
    set<int> s1;
    //插入元素
    s1.insert(10);
    s1.insert(30);
    s1.insert(40);
    s1.insert(20); //插入后自动排序
    printSet(s1);
    //删除
    s1.erase(s1.begin());
    printSet(s1);
    //删除重载的版本
    s1.erase(30);
    printSet(s1);
    //清除
    s1.clear(); //等价于s1.erase(s1.begin(), s1.end());
    printSet(s1);
    return 0;
}
```

set容器的大小与交换

size() 返回容器中元素的数目

empty() 判断容器是否为空

swap(st) 交换两个集合容器

set容器的大小与交换

```
#include<iostream>
#include<set>
using namespace std;
void printSet(set<int>& s) {
    for (auto it = s.begin(); it != s.end(); it++) {
        cout << *it << " ";
    }
    cout << endl;
}
int main() {
    set<int> s1={10,30,20,40};
    //打印容器
    printSet(s1);
    //判断容器是否为空
    if (s1.empty()) {
        cout << "s1为空" << endl;
    }
    else{
        cout << "s1不为空" << endl;
        cout << "s1的大小为:" << s1.size() << endl;
    }
    //交换容器
    set<int> s2 = {50,60,80,90,70};
    cout << "交换前:" << endl;
    printSet(s1);
    printSet(s2);

    s1.swap(s2); //交换容器
    cout << "交换后:" << endl;
    printSet(s1);
    printSet(s2);
    return 0;
}
```

set容器的查找与统计

<code>find(key)</code>	查找key是否存在，若存在返回该键的元素的迭代器；若不存在，返回 <code>set.end()</code>
<code>count(key)</code>	统计key的元素个数

set容器的查找与统计

```
#include<iostream>
#include <set>
using namespace std;
//set容器的查找与统计
int main() {
    set<int> s1;
    //插入数据
    s1.insert(20);
    s1.insert(30);
    s1.insert(10);
    s1.insert(40);
    //查找
    set<int>::iterator pos = s1.find(30);
    if (pos != s1.end()) { //找到元素
        cout << "找到该元素:" << *pos << endl;
    }
    else {
        cout << "未找到该元素" << endl;
    }
    //统计
    //这里需要注意 set容器不予许出现重复的元素
    //因此count返回的值只能为0或者1
    //统计30的个数
    s1.insert(30);
    s1.insert(30);
    int num = s1.count(30);
    cout << "30的个数为:" << num << endl; //1
    return 0;
}
```

set与multiset容器的区别

- Multiset是set集合容器的一种，其拥有set的全部内容，在此基础上，multiset还具备了可以重复保存元素的功能，因此会有略微和set的差别。
- Multiset容器在执行insert()时，只要数据不是非法数据和空数据，insert就总是能够执行，无论是一个数据还是一段数据。
- Multiset容器中的find()函数会返回和参数匹配的元素的迭代器，即使存在多个元素也只是返回第一个，如{10,20,20,20}搜索20进行匹配将会返回第二个参数，如果没有符合的参数则结束迭代器。
- 同理诸如lower_bound()等的需要进行一个位置的返回值，则统统返回第一个发现的值。

set与multiset容器的区别

```
#include <iostream>
#include <set>
#include <algorithm>
using namespace std;

int main(){
    multiset<int> ms;
    ms.insert(10);
    ms.insert(20);
    ms.insert(10);
    ms.insert(20);
    ms.insert(30);
    ms.insert(50);
    //{10,20,10,20,30,50} -----> {10,10,20,20,30,50} 插入时即会自动排序
    cout<< "发现20的位置在" << *ms.find(20)<<endl; //这样是找不出来的哟

    int i=0;
    for(multiset<int>::iterator it=ms.begin();it!=ms.find(20);it++,i++){
        cout<< "发现20的位置在" << i <<endl; //由于是从0开始计算的因此答案是2
    }

    return 0;
}
```

map/multimap容器

map容器中所有元素都是pair,pair中第一个元素为key（键值），起到索引作用，第二个元素为value（实值）。

同时，所有元素都会根据元素的键值自动排序。

map/multimap属于关联式容器，底层数据结构是用二叉树实现的。它的优点就是可以根据key值快速找到value值。

这里需要了解map与multimap的区别：

即map不予许容器中有重复的key值元素；而multimap允许容器中有重复的key值元素，这里的区别与set与multiset十分类似。

什么是Pair对组？

pair只含有两个元素，可以看作是只有两个元素的结构体。对于成对出现的数据，利用对组可以返回两个数据

应用：

代替二元结构体

作为map键值对进行插入

在创建pair对象时，必须提供两个类型名，两个对应的类型名的类型不必相同，可以在定义时进行成员初始化。

什么是Pair对组？

标准头文件 `#include<utility>`

vs里面，某些编译器可以不声明这个头文件而直接使用，貌似在C++中，pair被放入了std命名空间中了。

Pair初始化

格式为：

```
template <class T1, class T2> struct pair;
```

在现实情况中我们可以像类似于STL创建新容器一样创建pair也可以直接使用，如下：

```
pair<int,int> p;
```

```
pair<int,int> p(10,20);
```

或者是

```
map<char,int> m;
```

```
m.insert(pair<char,int>('a',10));
```

Pair数据访问

//数据类型 数据

```
pair<string, int>p("公孙离", 17);
```

```
cout << "姓名:" << p.first << " 年岁:" << p.second << endl;
```

//和结构体类似， first代表第一个元素， second代表第二个元素

```
pair<char, float>p1=make_pair('p',3.14 );
```

```
cout << "字符:" << p1.first << " 小数:" << p1.second << endl;
```

make_pair

函数原型 `template pair make_pair(T1 a, T2 b) { return pair(a, b); }`

可以通过 `make_pair` 生成我们所需要的 `pair`，对于一般的 `pair` 而言，如果需要对其进行赋值，则需要

```
pair<int,int> p;  
  
p.first=10,p.second=20;
```

但如果使用 `make_pair` 方法，则可以变成如下内容：

```
pair<int,int> p;  
  
p=make_pair(10,20);
```

可以看见，使用 `make_pair` 不仅仅让我们免去了对两个变量进行分开来的访问赋值，同时 `make_pair` 也智能的接受变量的类型，不需要再度指定，也就是说，`make_pair` 本身是接受隐式类型转换的，比如定义的是一个 `int` 类型，使用 `make_pair` 传入一个 `float` 类型的参数，`make_pair` 不会报错，而是回自动的进行一个类型转换，将 `float` 变为 `int`，这样可以获得更高的灵活度，同时也会有一些小问题。

map容器的构造与赋值

`map<T,T> m`

`map(const map &mp)`

`map& operator=(const map &mp)`

map默认构造函数

拷贝构造函数

重载等号操作符

map容器的构造与赋值

```
#include<iostream>
#include<map>
using namespace std;
//map容器的构造与赋值
void printMap(map<int, int>& m) {
    for (auto it = m.begin(); it != m.end(); it++) {
        cout << "key =" << it->first << " value =" << it->second << endl;
    }
    cout << endl;
}
int main() {
    //创建map容器
    map<int, int> m;
    //插入数据里面需要传入的是对组
    m.insert(pair<int, int>(1, 10)); //pair<int, int>(1, 10)为匿名二元组
    m.insert(pair<int, int>(3, 30));
    m.insert(pair<int, int>(4, 40));
    m.insert(pair<int, int>(2, 20));
    //插入元素后会根据key自动进行升序排列
    printMap(m);

    //拷贝构造
    map<int, int> m2(m);
    printMap(m2);

    //赋值
    map<int, int> m3;
    m3 = m2;
    printMap(m3);
    return 0;
}
```

map容器的大小与交换

size() 返回容器中元素的数目

empty() 判断容器中是否为空

swap(st) 交换两个集合容器

map容器的大小与交换

```
#include<iostream>

#include<map>

using namespace std;

//map容器的大小与交换

void printMap(map<int, int>& m) {
    for (auto it = m.begin(); it != m.end(); it++) {
        cout << "key值为:" << it->first << " value值为:" << it->second << endl;
    }
    cout << endl;
}
```

map容器的大小与交换

```
int main() {  
    map<int, int> m;  
    m.insert(pair<int, int>(1, 10));  
    m.insert(pair<int, int>(3, 30));  
    m.insert(pair<int, int>(2, 20));  
    if (m.empty()) {  
        cout << "m容器为空" << endl;  
    }  
    else {  
        cout << "m容器不为空" << endl;  
        cout << "m的大小为:" << m.size() << endl;  
    }  
    //交换  
    map<int, int> m2;  
    m2.insert(pair<int, int>(4, 40));  
    m2.insert(pair<int, int>(6, 60));  
    m2.insert(pair<int, int>(5, 50));  
    cout << "map容器交换前" << endl;  
    printMap(m);  
    printMap(m2);  
    cout << "map容器交换后" << endl;  
    m.swap(m2);  
    printMap(m);  
    printMap(m2);  
    return 0;  
}
```

map容器的插入与删除

<code>insert(elem)</code>	在容器中插入元素
<code>clear()</code>	清除所有元素
<code>erase(pos)</code>	删除pos迭代器所指的元素，返回下一个元素的迭代器
<code>erase(beg,end)</code>	删除区间[beg,end)的所有元素，返回下一个元素的迭代器
<code>erase(key)</code>	删除容器中值为key的元素

map容器的插入与删除

```
#include<iostream>
#include <map>
using namespace std;
int main() {
    map<int, int> m;
    m.insert(pair<int, int>(1, 10)); //插入 第一种

    m.insert(make_pair(2, 20)); //插入 第二种

    m.insert(map<int, int>::value_type(3, 30)); //插入 第三种 不建议使用

    //插入 第四种 最简单
    //[]不建议插入 通过[]可以利用key访问到value
    //使用[]插入元素的时候,如果key不存在将会自动创建键值对
    m[4] = 40;
    printMap(m);

    m.erase(m.begin()); //删除
    printMap(m);

    m.erase(3); //删除 直接传入key
    printMap(m);
    //全部删除
    m.clear(); //相当于m.erase(m.begin(), m.end())
    printMap(m);
    return 0;
}
```

map容器的查找与统计

<code>find(key)</code>	查找key是否存在，返回该键的元素的迭代器；若不存在返回 <code>map.end()</code>
<code>count(key)</code>	统计key的元素的个数

map容器的查找与统计

```
#include<iostream>
#include<map>
using namespace std;
int main() {
    //查找
    map<int, int> m;
    m.insert(pair<int, int>(1, 10));
    m.insert(pair<int, int>(3, 30));
    m.insert(pair<int, int>(2, 20));
    //查找键为3的键值对
    map<int,int>::iterator pos=m.find(3);
    if (pos != m.end()) {
        cout << "查到了元素 key=" << pos->first << " value=" << pos->second << endl;
    }
    else {
        cout << "未找到元素" << endl;
    }
    //统计
    //由于map容器中key不能重复出现 因此count统计的结果只有0或1
    int num=m.count(3);//返回结果为整型
    cout << "num=" << num << endl;
    return 0;
}
```

Multimap容器

Multimap是map映射容器的一种，其拥有map的全部内容，并在此基础上，multimap还具有了可以重复保存元素的功能，与上文的multiset差不多，任何进行访问单个值得语句访问均只会返回第一个位置，这里不再多说，而是举一个实际中可能用得到的例子。

有没有一种方法，使得一个key值能够对应多个value，产生一种诸如一个学生有多门考试成绩一样的映射

我们都知道map关联容器是使得一个数据与另一个数据发生映射的容器，通过key得到value产生一一对应，那么multimap在此基础上使得map元素可以重复，因此这种情况可以使用multimap

Multimap容器

```
#include <iostream>
#include <string>
#include <map>
using namespace std;

int main() {
    multimap<string, int> m_map;
    string name = "XiaoMing";
    m_map.insert(make_pair(name, 50));
    m_map.insert(make_pair(name, 55));
    m_map.insert(make_pair(name, 60));
    m_map.insert(make_pair("zhangsan", 30));
    //查找name并打印
    int k;
    multimap<string, int>::iterator m;
    m = m_map.find(name);
    for (k = 0; k != m_map.count(name); k++, m++)
        cout << m->first << "--" << m->second << endl;
    return 0;
}
```

STL常用算法

算法主要是由头文件<algorithm><functional><numeric>组成

- 1.<algorithm>是所有STL头文件中最大的一个，范围涉及到比较、交换、查找、遍历操作、复制、修改等等
- 2.<numeric>体积很小，只包括几个再序列上面进行简单数学运算的模板函数
- 3.<functional>定义了一些模板类，用以声明函数对象

STL常用遍历算法

`for_each` //遍历容器

`transform` //搬运容器到另一个容器中

for_each

功能描述：实现遍历容器

函数原型：

```
for_each(iterator beg, iterator end, _func);
```

//遍历算法 遍历容器元素

//beg 开始迭代器

//end 结束迭代器

//_func 函数或者函数对象

for_each

```
#include<iostream>
using namespace std;
#include<vector>
#include<algorithm>
//常用遍历算法 for_each
```

//普通函数

```
void print01(int val)
{
    cout << val << " ";
}
```

//仿函数

```
class print02
{
public:
    void operator()(int val)
    {
        cout << val << " ";
    }
};
```

```
void test01()
{
    vector<int>v;
    for (int i = 0; i < 10; i++)
    {
        v.push_back(i);
    }

    for_each(v.begin(), v.end(), print01);
    cout << endl;

    for_each(v.begin(), v.end(), print02());
    cout << endl;
}

int main() {
    test01();
    return 0;
}
```

transform

功能描述：搬运容器到另一个容器中

函数原型：

○transform(iterator beg1, iterator end1, iterator beg2, _func);

//beg1 源容器开始迭代器

//end1 源容器结束迭代器

//beg2 目标容器开始迭代器

//_func 函数或者函数对象

transform

```
#include<iostream>
using namespace std;
#include<vector>
#include<algorithm>
//常用遍历算法 transform
class Transform
{
public:
    int operator()(int v)
    {
        return v + 100;
    }
};
class Myprint
{
public:
    void operator()(int val)
    {
        cout << val << " ";
    }
};
```

```
void test01(){
    vector<int> v;
    for (int i = 0; i < 10; i++) {
        v.push_back(i);
    }
    vector<int> vTarget;//目标容器
    vTarget.resize(v.size()); //目标容器需要提前开辟空间
    transform(v.begin(), v.end(), vTarget.begin(), Transform());

    for_each(vTarget.begin(), vTarget.end(), Myprint());
    cout << endl;
}
int main() {
    test01();
    return 0;
}
```

stl常用查找算法

find //查找元素

find_if //按条件查找元素

adjacent_find //查找相邻重复元素

binary_search //二分查找法

count //统计元素个数

count_if //按条件统计元素个数

find

功能描述：查找指定元素，找到返回指定元素的迭代器，找不到返回结束迭代器end()

函数原型：

```
find(iterator beg, iterator end, value);
```

//按值查找元素，找到返回指定位置迭代器，找不到返回结束迭代器位置

//beg 开始迭代器

//end 结束迭代器

//value 查找的元素

find

```
vector<int> v;  
for (int i = 0; i < 10; i++)  
{  
    v.push_back(i);  
}  
  
//查找容器中是否有9这个元素  
vector<int>::iterator it = find(v.begin(), v.end(), 9);  
if(it == v.end())  
{  
    cout << "not found " << endl;  
}  
else  
{  
    cout << " found: " << *it << endl;  
}
```

find_if

功能描述：按条件查找元素

函数原型：

```
find_if(iterator beg, iterator end, _Pred);
```

//按值查找元素，返回指定位置迭代器，找不到返回结束迭代器位置

//beg 开始迭代器

//end 结束迭代器

// _Pred 函数或者谓词（返回bool类型的仿函数）

find_if

```
#include<iostream>
using namespace std;
#include<vector>
#include<algorithm>
#include<string>
//常用查找算法
//find_if
class GreaterFive
{
public:
    bool operator()(int val)
    {
        return val == 5;
    }
};
```

```
void test01(){
    vector<int> v;
    for (int i = 0; i < 10; i++)
    {
        v.push_back(i);
    }

    //查找容器中是否有5这个元素
    vector<int>::iterator it = find_if(v.begin(), v.end(), GreaterFive());
    if (it == v.end())
    {
        cout << "not found " << endl;
    }
    else
    {
        cout << " found: " << *it << endl;
    }
}

int main() {

    test01();
    return 0;
}
```

adjacent_find

功能描述：查找相邻重复元素

函数原型：

```
adjacent_find(iterator beg, iterator end);
```

//查找相邻重复元素，返回相邻元素的第一个位置的迭代器

//beg 开始迭代器

//end 结束迭代器

adjacent_find

```
#include<iostream>
using namespace std;
#include<vector>
#include<algorithm>
//常用查找算法
//adjacent_find
void test01()
{
    vector<int> v={0,2,2,1,3,4,4};

    vector<int>::iterator pos = adjacent_find(v.begin(), v.end());
    if (pos == v.end())
    {
        cout << "not found " << endl;
    }
    else
    {
        cout << "found : " << *pos << endl;
    }
}

int main() {
    test01();
    return 0;
}
```

binary_search

功能描述：查找指定元素是否存在

函数原型：

○ `bool binary_search(iterator beg, iterator end, value);`

//查找指定的元素， 查到 返回true 否则false

//注意： 在无序序列中不可用

//beg 开始迭代器

//end 结束迭代器

//value 查找的元素

binary_search

```
#include<iostream>
using namespace std;
#include<vector>
#include<algorithm>
//常用查找算法 binary_search
void test01()
{

    vector<int> v;
    for (int i = 0; i < 10; i++)
    {
        v.push_back(i);
    }
    // v.push_back(2);//如果是无序序列，结果未知
    //查找容器中是否有9这个元素
    //注意：容器必须是有序的序列
    bool ret = binary_search(v.begin(), v.end(), 9);
    if (ret)
    {
        cout << "found" << endl;
    }
    else
    {
        cout << "not found" << endl;
    }
}

int main() {
    test01();
    return 0;
}
```

count

功能描述：统计元素个数

函数原型：

```
count(iterator beg,iterator end, value);
```

//统计元素出现次数

//beg 开始迭代器

//end 结束迭代器

//value 统计的元素

count

```
vector<int> v;
```

```
    v.push_back(10);
```

```
    v.push_back(40);
```

```
    v.push_back(20);
```

```
    v.push_back(40);
```

```
    v.push_back(20);
```

```
    v.push_back(40);
```

```
//统计容器中40元素有几个
```

```
int num = count(v.begin(), v.end(), 20);
```

```
cout << "40's num is :" << num << endl;
```

count_if

功能描述：按条件统计元素个数

函数原型：

```
count_if(iterator beg, iterator end, _Pred);
```

//按条件统计元素出现次数

//beg 开始迭代器

//end 结束迭代器

//_Pred 谓词

count_if

```
#include<iostream>
using namespace std;
#include<vector>
#include<algorithm>
#include<string>
//常用查找算法
//count_if

class Greater10
{
public:
    bool operator()(int val)
    {
        return val > 10;
    }
};
```

```
int main()
{
    vector<int> v;
    v.push_back(10);
    v.push_back(40);
    v.push_back(30);
    v.push_back(40);
    v.push_back(20);
    v.push_back(40);

    //统计容器中大于10元素有几个
    int num = count_if(v.begin(), v.end(), Greater10());

    cout << "greater than 10's num is :" << num << endl;

    return 0;
}
```

stl常用排序算法

sort //对容器内元素进行排序

random_shuffle //洗牌 指定范围内的元素随机调整次序

merge //容器元素合并，并存储到另一容器中

reverse //反转指定范围的元素

sort

功能描述：对容器内元素进行排序

函数原型：

```
sort(iterator beg, iterator end, _Pred);
```

//按值查找元素，找到返回指定位置迭代器，找不到返回结束迭代器位置

//beg 开始迭代器

//end 结束迭代器

//_Pred 谓词

sort

```
#include<iostream>
using namespace std;
#include<algorithm>
#include<vector>
#include<functional>
//常用排序算法 sort

void myPrint(int val)
{
    cout << val << " ";
}
```

```
void test01()
{
    vector<int> v={2,5,1,3,4,6};

    //利用sort进行升序
    sort(v.begin(), v.end());
    for_each(v.begin(), v.end(), myPrint);
    cout << endl;

    //利用sort进行降序
    sort(v.begin(), v.end(), greater<int>());
    for_each(v.begin(), v.end(), myPrint);
    cout << endl;
}

int main() {
    test01();
    return 0;
}
```

random_shuffle

功能描述：洗牌 指定范围内的元素随机调整次序

函数原型：

```
random_shuffle(iterator beg, iterator end);
```

//指定范围内的元素随机调整次序

//beg 开始迭代器

//end 结束迭代器

random_shuffle

```
#include<iostream>
using namespace std;
#include<vector>
#include<algorithm>
#include<ctime>
//常用排序算法 random_shuffle
void myPrint(int val)
{
    cout << val << " ";
}
void test01()
{
    srand((unsigned int)time(NULL));
    vector<int> v={1,2,3,4,5};

    //利用洗牌打乱顺序
    random_shuffle(v.begin(), v.end());
    for_each(v.begin(), v.end(), myPrint);
    cout << endl;
}
int main() {
    test01();
    return 0;
}
```

merge

功能描述：两个容器元素合并，并存储到另一个容器中

函数原型：

```
merge(iterator beg1, iterator end1, iterator beg2, iterator end2, iterator dest);
```

//容器元素合并，并存储到另一个容器中

//注意：两个容器必须是有序的

//beg1 容器1开始迭代器

//end1 容器1结束迭代器

//beg2 容器2开始迭代器

//end2 容器2结束迭代器

//dest 目标容器开始迭代器

merge

```
#include<iostream>
using namespace std;
#include<vector>
#include<algorithm>

//常用排序算法 merge
void myPrint(int val)
{
    cout << val << " ";
}

void test01()
{
    vector<int> v1={1,3,5,9,11};
    vector<int> v2={2,4,8,12};
    //目标容器
    vector<int> vTarget;

    //提前给目标容器分配空间
    vTarget.resize(v1.size() + v2.size());

    merge(v1.begin(), v1.end(), v2.begin(), v2.end(), vTarget.begin());
    for_each(vTarget.begin(), vTarget.end(), myPrint);
    cout << endl;
}

int main() {
    test01();
    return 0;
}
```

reverse

功能描述：将容器内元素进行反正

函数原型：

```
reverse(iterator beg, iterator end);
```

//反转指定范围的元素

//beg 开始迭代器

//end 结束迭代器

reverse

```
#include<iostream>
using namespace std;
#include<vector>
#include<algorithm>

//常用排序算法 reverse
void myPrint(int val){
    cout << val << " ";
}
void test01(){
    vector<int> v={1,3,5,6,9};

    cout << "before reverse" << endl;
    for_each(v.begin(), v.end(), myPrint);
    cout << endl;
    cout << "after reverse" << endl;
    reverse(v.begin(), v.end());
    for_each(v.begin(), v.end(), myPrint);
    cout << endl;
}
int main() {
    test01();
    return 0;
}
```

常用拷贝和替换算法

`copy` //容器内指定范围的元素拷贝到另一个容器中

`replace` //将容器内指定范围的旧元素修改为新元素

`replace_if` //容器内指定范围满足条件的元素替换为新元素

`swap` //交换两个容器的元素

copy

功能描述： 容器内指定范围的元素拷贝到另一个容器中

函数原型：

```
copy(iterator beg, iterator end, iterator dest);
```

//将开始迭代器到结束迭代器之间的元素拷贝到目标容器中

//beg 开始迭代器

//end 结束迭代器

//dest 目标开始迭代器

copy

```
#include<iostream>
using namespace std;
#include<vector>
#include<algorithm>

//常用拷贝和替换算法 copy
void myPrint(int val){
    cout << val << " ";
}
void test01(){
    vector<int> v1 = { 11,22,33,55,66,99 };
    vector<int> v2;
    v2.resize(v1.size());

    copy(v1.begin(), v1.end(), v2.begin());
    for_each(v2.begin(), v2.end(), myPrint);
    cout << endl;
}
int main() {
    test01();
    return 0;
}
```

replace

功能描述：将容器内指定范围的旧元素替换为新元素

函数原型：

```
replace(iterator beg, iterator end, oldvalue, newvalue);
```

//将区间内旧元素 替换成新元素

//beg 开始迭代器

//end 结束迭代器

//oldvalue 旧元素

//newvalue 新元素

replace

```
#include<iostream>
using namespace std;
#include<vector>
#include<algorithm>
//常用拷贝和替换算法 replace
class MyPrint
{
public:
    void operator()(int val){
        cout << val << " ";
    }
};
void test01(){
    vector<int> v={10,20,10,30,10,40};
    cout << "before replace " << endl;
    for_each(v.begin(), v.end(), MyPrint());
    cout << endl;

    //将10 替换成1000
    replace(v.begin(), v.end(), 10, 1000);
    cout << "after replace " << endl;
    for_each(v.begin(), v.end(), MyPrint());
    cout << endl;
}
int main() {
    test01();
    return 0;
}
```

replace_if

功能描述：将区间内满足条件的元素，替换成指定元素

函数原型：

```
replace_if(iterator beg, iterator end, _pred, newvalue);
```

//按条件替换元素，满足条件的替换成指定元素

//beg 开始迭代器

//end 结束迭代器

//_pred 谓词

//newvalue 替换的新元素

replace_if

```
#include<iostream>
using namespace std;
#include<vector>
#include<algorithm>

//常用拷贝和替换算法 replace_if
class MyPrint{
public:
    void operator()(int val)
    {
        cout << val << " ";
    }
};
class Greater20{
public:
    bool operator()(int val)
    {
        return val == 20;
    }
};
```

```
void test01()
{
    vector<int> v={1,10,20,30,20,50,10};

    cout << "before replace " << endl;
    for_each(v.begin(), v.end(), MyPrint());
    cout << endl;

    //大于20 替换成1000
    replace_if(v.begin(), v.end(), Greater20(), 1000);
    cout << "after replace " << endl;
    for_each(v.begin(), v.end(), MyPrint());
    cout << endl;
}

int main() {
    test01();
    return 0;
}
```

swap

功能描述： 互换两个容器的元素

函数原型：

```
swap(container c1, container c2);
```

```
//互换两个容器的元素
```

```
//c1 容器1
```

```
//c2 容器2
```

swap

```
#include<iostream>
using namespace std;
#include<vector>
#include<algorithm>

//常用拷贝和替换算法 swap
void myPrint(int val){
    cout << val << " ";
}

void test01(){
    vector<int> v1={1,2,3};
    vector<int> v2={11,22,33};

    cout << "before swap " << endl;
    for_each(v1.begin(), v1.end(), myPrint);
    cout << endl;
    for_each(v2.begin(), v2.end(), myPrint);
    cout << endl;
    cout << "after swap " << endl;
    swap(v1, v2);
    for_each(v1.begin(), v1.end(), myPrint);
    cout << endl;
    for_each(v2.begin(), v2.end(), myPrint);
    cout << endl;
}

int main() {
    test01();
    return 0;
}
```

常用算术生成算法

`accumulate` //计算容器元素累计总和

`fill` //向容器中添加元素

算术生成算法属于小型算法，使用时包含的头文件为 `#include<numeric>`

accumulate

accumulate

英[ə'kju:mjələit]

功能描述：计算区间内 容器元素累计总和

函数原型：

`accumulate(iterator beg, iterator end, value);`

//计算容器元素累计总和

//beg 开始迭代器

//end 结束迭代器

//value 起始值

accumulate

```
#include<iostream>
using namespace std;
#include<vector>
#include<numeric>

//常用算术生成算法
void test01(){
    vector<int> v;
    for (int i = 0; i <= 100; i++)
    {
        v.push_back(i);
    }
    int total = accumulate(v.begin(), v.end(), 0); //参数3 起始累加值

    cout << " total = " << total << endl;
}
int main() {
    test01();
    return 0;
}
```

fill

功能描述：向容器中填充指定的元素

函数原型：

```
fill(iterator beg, iterator end, value);
```

//向容器中填充元素

//beg 开始迭代器

//end 结束迭代器

//value 填充值

fill

```
#include<iostream>
using namespace std;
#include<vector>
#include<numeric>
#include<algorithm>
//常用算术生成算法 fill
void myPrint(int val){
    cout << val << " ";
}
void test01(){
    vector<int> v;
    v.resize(10);

    //后期重新填充
    fill(v.begin(), v.end(), 100);
    for_each(v.begin(), v.end(), myPrint);
    cout << endl;
}
int main() {
    test01();
    return 0;
}
```

常用集合算法

`set_intersection` //求两个容器的交集

`set_union` //求两个容器的并集

`set_difference` //求两个容器的差集

set_intersection

功能描述：求两个容器的交集

函数原型：

```
set_intersection(iterator beg1, iterator end1, iterator beg2, iterator end2, iterator dest);
```

//求两个容器的交集

//注意：两个集合必须是有序序列

//beg1 容器1开始迭代器

//end1 容器1结束迭代器

//beg2 容器2开始迭代器

//end2 容器2结束迭代器

//dest 目标容器开始迭代器

set_intersection

```
#include<iostream>
using namespace std;
#include<vector>
#include<algorithm>
//常用集合算法 set_intersection
void myPrint(int val){
    cout << val << " ";
}
void test01(){
    vector<int> v1={1,2,3,4,5};
    vector<int> v2={1,3,6,9};
    vector<int> vTarget;
    //目标容器需要提前开辟空间
    //最特殊情况 大容器包含小容器 开辟空间 去小容器的size即可
    vTarget.resize(min(v1.size(), v2.size()));

    //获取交集
    vector<int>::iterator itEnd = set_intersection(v1.begin(), v1.end(), v2.begin(), v2.end(), vTarget.begin());

    for_each(vTarget.begin(), itEnd, myPrint);
    cout << endl;
}
int main() {
    test01();
    return 0;
}
```

求交集的两个集合必须是有序序列

目标容器开辟空间需要从两个容器中去小值

set_intersection返回值即是交集中最后一个元素的位置

set_union

功能描述：求两个集合的并集

函数原型：

```
set_union(iterator beg1, iterator end1, iterator beg2, iterator end2, iterator dest);
```

//求两个集合的并集

//注意：两个集合必须是有序序列

//beg1 容器1开始迭代器

//end1 容器1结束迭代器

//beg2 容器2开始迭代器

//end2 容器2结束迭代器

//dest 目标容器开始迭代器

set_union

```
#include<iostream>
using namespace std;
#include<vector>
#include<algorithm>
//常用集合算法 set_union
void myPrint(int val){
    cout << val << " ";
}
void test01(){
    vector<int> v1={1,2,3};
    vector<int> v2={4,5,6};

    vector<int> vTarget;
    //目标容器需要提前开辟空间
    //最特殊情况 两个容器没有交集, 并集就是两个容器size相加
    vTarget.resize(v1.size() + v2.size());

    //获取交集
    vector<int>::iterator itEnd = set_union(v1.begin(), v1.end(), v2.begin(), v2.end(), vTarget.begin());

    for_each(vTarget.begin(), itEnd, myPrint);
    cout << endl;
}
int main() {
    test01();
    return 0;
}
```

set_difference

功能描述：求两个集合的差集

函数原型：

```
set_difference(iterator beg1, iterator end1, iterator beg2, iterator end2, iterator dest);
```

//求两个集合的差集

//注意：两个集合必须是有序序列

//beg1 容器1开始迭代器

//end1 容器1结束迭代器

//beg2 容器2开始迭代器

//end2 容器2结束迭代器

//dest 目标容器开始迭代器

set_difference

```
#include<iostream>
using namespace std;
#include<vector>
#include<algorithm>
//常用集合算法 set_difference
void myPrint(int val){
    cout << val << " ";
}
void test01(){
    vector<int> v1={1,2,3,4,5};
    vector<int> v2={2,4,5,6,8};

    vector<int> vTarget;
    //目标容器需要提前开辟空间
    //最特殊情况 两个容器相交 开辟空间 取大容器的size即可

    vTarget.resize(max(v1.size(), v2.size()));

    cout << "v1 and v2's difference set is : " << endl;
    vector<int>::iterator itEnd = set_difference(v1.begin(), v1.end(), v2.begin(), v2.end(), vTarget.begin());
    for_each(vTarget.begin(), itEnd, myPrint);
    cout << endl;

    cout << "v2 and v1's difference set is : " << endl;
    itEnd = set_difference(v2.begin(), v2.end(), v1.begin(), v1.end(), vTarget.begin());
    for_each(vTarget.begin(), itEnd, myPrint);
    cout << endl;
}
int main() {
    test01();
    return 0;
}
```

练习题目

约瑟夫问题

共 n 个人，每次数到第 m 个人时，这个人则会出圈，出圈后会再次从1开始数数，直至所有人都出圈，要求按顺序输出出圈人的编号。

解题思路：可以想到队列，先进行计数是第几个人，在未到第 m 个人时先读入再弹出，直至到第 m 个人输出第 m 个人的编号，再次从头开始计数，如此循环

练习代码

```
#include<iostream>
using namespace std;
#include<queue>//队列所需头文件
queue<int>q;//读入的数, int类型
int n, m, num = 1;//n人, m时出圈, bs报数
int main()
{
    cin >> n >> m;
    for (int i = 1; i <= n; i++)
    {
        q.push(i);//先把数放入队列中
    }
    while (!q.empty())//队列非空时
    {
        if (num == m)//报数等于m需出圈
        {
            cout << q.front() << " ";//先输出
            q.pop();//再弹出
            num = 1;//重新计数
        }
        else//报数未到 m 时
        {
            num++;//报数++
            q.push(q.front());//将队首放入
            q.pop();//弹出
        }
    }
    return 0;
}
```

知识点总结

1. STL概述
2. STL基本组成
3. string容器
4. vector容器
5. stack容器
6. queue容器
7. deque容器
8. list容器
9. set/multiset容器
10. map/multimap容器
11. STL常用算法

了解

熟悉

重点

重点

重点

重点

重点

重点

重点

重点

重点

作业

假设一个表达式有英文字母（小写）、运算符（+，-，*，/）和左右小（圆）括号构成，以“@”作为表达式的结束符。请编写一个程序检查表达式中的左右圆括号是否匹配，若匹配，则返回“YES”；否则返回“NO”。假设表达式长度小于255，左圆括号少于20个。

【输入】

一行数据，即表达式。

【输出】

一行，即“YES”或“NO”。

【输入样例】

$2*(x+y)/(1-x)@$

【输出样例】

YES

下节课预告

文件操作